Interview questions Python **4**

1. To Prepare 4
2. Setup, Installation and Versions 5
    1. How to install Python on Windows and set path variable? 5
3. Python OOPS 5
    1. Does Classes use memory? - No 5
    2. Abstract class? 5
    3. Overload constructor in a class? - Yes 5
    4. Overload destructor in a class? - No 5
    5. Destructor? 5
    6. Constructor? 5
    7. Access specifiers 5
    8. Polymorphism? and types of Polymorphism? 6
4. Python Fundamentals & Memory 6
    1. What is Python? What are the benefits of using Python? 6
    2. Why we use Python? 6
    3. What type of language is python? Programming or scripting? 7
    4. How Python is interpreted? 7
    5. Is Python, case sensitive language or not? 7
    6. How memory is managed in Python? 8
    7. What are the key features of Python? 8
    8. Why we create virtual environment in Python, venv vs Docker? 8
    9. What is pickling and unpickling? 8
    10. What is PEP 8? 9
    11. Why indentation is required in Python? 9
    12. What are Python namespaces? 9
5. Programming CORE 10
    1. Is it possible to return two or more values in Python? 10
    2. What is abstract in python? 10
    3. Mutable vs Immutable object difference 11
    4. What is Static method and Class methods and their differences. 11
    5. Multiprocess, Multithreading, AsyncIO 12
    6. A Global Interpreter Lock (GIL) 13
    7. Debugging skills 14
    8. Singleton class 14
    9. Monkey patching 14

10. What is the difference between local and global variables? 14
11. How can you share global variables across modules? 14
12. What is Keyword? 15
13. What is Float, int() function, print? 15
14. How can you convert a number to a string? 15
15. How to convert JSON into Python object 15
6. Python Fundamentals 15
1. Whenever Python exits, why isn't all the memory de-allocated 15
2. What is PYTHONPATH? 15
3. What are Assignment, Identity operators & Membership operators operators? 15
4. A What is the maximum possible length of an identifier? 17
5. Property in Python. 17
6. Assembly 17
7. What is Input? (The input() function) 17
8. What is “if statement”? 17
9. For VS While Loop 17
10. What are Iterator & generators in Python? 18
11. What is the difference between Xrange and range? 19
12. What is decorator with code examples? Meta-programming (Decorators) 19
13. Middleware & CORS 20
14. Middleware (Runs before/after request): 20
15. CORS Concepts (Cross-Origin Resource Sharing) 20
16. Implementing CORS in Python Frameworks 21
17. Key Considerations & Best Practices 21
18. A What is 'with' keyword in python? 21
19. What is memory leaks, Diagnose and Fix Memory leaks 21
20. How to Diagnose Memory Leaks in Python 22
21. Async and await 22
22. What is meant by jump statement? 23
23. What are the tools that help to find bugs or perform static analysis? 23
24. How are arguments passed by value or by reference 23
25. What is lambda in Python? 23
26. Why lambda forms in python does not have statements 23
27. What is pass in Python? 23
28. What is docstring in Python? 24
29. What is negative index in Python? 24

30. How you can convert a number to a string? 24
31. What is module and package in Python? 24
32. Mention what are the rules for local and global variables in Python? 24
33. How can you share global variables across modules? 24
34. Explain how can you make a Python Script executable on Unix? 24
35. Explain how to delete a file in Python? 24
36. Explain how can you generate random numbers in Python? 24
37. Explain how can you access a module written in Python from C? 24
38. How can you copy an object in Python? 24
39. Shallow copy VS Deep copy using code example. 25
42. Positional and Keyword Arguments 27
43. What is comment, its types? 27
44. What is Keyword? 27
45. What is Float, int() function, print? 27
46. What is type conversion in Python? 27
47. Package VS Library 28
48. What is first class objects in Python? 28

7. LIST TUPLE DICTIONARY AND SET 29
    1. What are the built-in data type does python provides 29
    2. What is the difference between list and tuples in Python? 29
    3. When to use tuple? 29
    4. Why Tuple is immutable? 29
    5. Can we add an elements in tuple, why, how? 29
    6. List, dict, comprehension 30
    7. Dictionary Questions: 31

8. File Handling & Exceptions 31
    1. What is file handling in python? 31
    2. What is file operation in python? (Opening, Writing, Appending, Reading, Closing) 31
    3. What is read ( ) function in file handling? 31
    4. How do you open a file for writing? (The old list had "how to delete," but not opening/writing) 31
    5. What are Reasons of Exception? 31
    6. What is try, except, finally block? 32
    7. Explain important Python errors? (Arithmetic Error, Import Error, Index Error) 32

9. UNIT TEST 33
    1. Beginner-Level Questions 33
        1. What is Pytest and why is it used? 33

4. What is unit test in Python? 34
10. Lists & Sequences 34
1. What is slicing? 34
2. What is pop ( ) and clear ( ) function in list? Removing Elements 34
3. What is sort( ) vs reverse ( ) vs count ( ) function in list? 34
4. What is append( ) vs extend( ) vs insert ( ) function in list? 34
5. How can you count duplicate elements in a given list? 35
11. Libraries 35
12. Big Libraries 36
13. Tools 36
14. Python Fields 36
2. Framework 37
3. Data Analysis 37
4. Data Visualization 37
5. Database / Big Data SQL language, Data Integration 37
6. Business Intelligence 37
7. Advanced Databases 37
8. Extraction and management of Data 37
9. Machine Learning 37
15. Miscellaneous & Coding Snippets 37
16. JPMC Interview Questions 37
1. 4. JPMC, 27th Feb 2026 37
2. 3. JPMC, 9th Jan 2026 38
3. 2. JPMC, Dot NET., 18th Dec 2026 38
4. [Java] Ashwini 38
5. [Java] Poornima 38
6. Shared by Neha 38
7. Python Programs 39
8. Most Imp Topics 39
17. Agents 40

Interview questions Python

1. To Prepare

1. Already asked questions
2. Questions from Internet
3. Past Project
4. Python core concepts
5. OOPs

6. File management - CSV Excel Json libraries for file management
7. AWS - CI/CD, Few common services
8. FastAPI
9. SQL
10. DSA
    1. Data structures - String, Array, LinkedList, Stack, Queue
    2. Algorithms - Kadane's, Sliding window, Binary Search
    3. Questions - Palindrome, Armstrong, Fibonacci, Anagram, Buy Sell stock, Rain water tapping, Target sum / two sum

2. Setup, Installation and Versions

1. How to install Python on Windows and set path variable?

- Download: Go to python.org and download the Windows installer.
- Install: Run the .exe file. Crucial: Check the box that says "Add Python to PATH" before clicking "Install Now."
- Manual Path Setup (if missed):
  - Right-click "This PC" > Properties > Advanced System Settings > Environment Variables.
  - Under "System Variables," find "Path" and click Edit.
  - Add the path to your Python installation (e.g., C:\Python39) and the Scripts folder (e.g., C:\Python39\Scripts).

3. Python OOPS

1. Does Classes use memory? - No

2. Abstract class?

Intended to be used for inheritance. can't be instantiated

3. Overload constructor in a class? - Yes

4. Overload destructor in a class? - No

5. Destructor?

method that is automatically called when the object goes out of scope or destroyed. In Python, the destructor is named **__del__**.

6. Constructor?

1. Block of code that initializes the newly created object.
2. A constructor resembles an instance method but it's not a method as it doesn't have a return type.
3. It generally is the method having the same name as the class but in some languages, it might differ.
   - For example: In python, a constructor is named **__init__**

7. Access specifiers

1. Special types of keywords that are used to specify or control the accessibility of entities like classes, methods, and so on.
2. **Private, Public, and Protected**

1. The key components of OOPs, encapsulation and data hiding, are largely achieved because of these access specifiers.

8. Polymorphism? and types of Polymorphism?

1. Many forms. It is the property of some code to behave differently for different contexts.
    1. Compile Time Polymorphism
    2. Runtime Polymorphism

4. Python Fundamentals & Memory

1. What is Python? What are the benefits of using Python?

- Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum in 1991. It is designed to be simple, readable, and highly expressive, allowing developers to write fewer lines of code compared to languages like Java or C++
  Python supports multiple programming paradigms:
  - Object-oriented - Procedural - Functional

- Benefits
  1. **Readable syntax** – almost like English, which improves maintainability.
  2. **Platform independent** – same code runs on Windows, Linux, Mac.
  3. **Free and open-source** – Python is free to use and distribute. It is open source.
  4. **Huge ecosystem** – Libraries for AI, ML, web, automation, data science.
  5. **Extensive support for libraries** – Python has vast libraries that contain almost any function needed. It also further provides the facility to import other packages using Python Package Manager(pip). Python comprises of a huge standard library for most Internet platforms like Email, HTML, etc.
  6. **Dynamic typing** – no need to declare variable type.
  7. **Automatic memory management** – garbage collector handles memory. Python does not require explicit memory management as the interpreter itself allocates the memory to new variables and free them automatically
  8. **Open source** – free and community supported.
  9. **Rapid development** – fewer lines of code → faster delivery.
  10. **Interpreted language –** Since python is interpreted language, it executes the code line by line and stops if an error occurs in any line.
  11. **Portable –** Python programs can run on any platform without requiring any change
  12. 👉 Interview tip: Python is often chosen for rapid prototyping and backend systems where development speed matters more than raw performance

2. Why we use Python?

- Python is used because it balances simplicity + power + versatility.
  1. Developers choose Python when they want:
     1. Fast development
     2. Easy maintenance
     3. Readable code
     4. Huge library support
  2. Companies use Python for:
     1. Web Application
     2. Software Development
     3. Database
     4. GUI application
     5. Scientific and Numeric Computing
     6. Business Applications
     7. Console Based Application
     8. Backend APIs
     9. Automation

10. Data pipelines
11. AI/ML
12. DevOps scripts

3. What type of language is python? Programming or scripting?

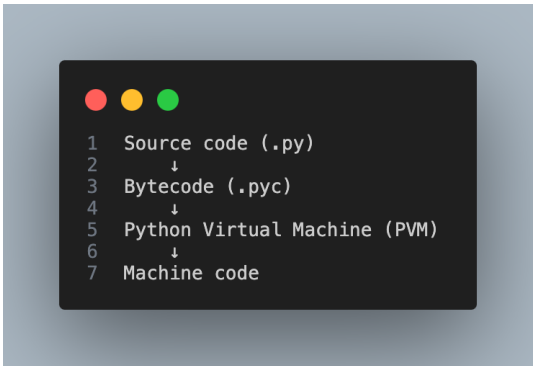
- Python is both. Technically it is a general-purpose programming language, but it is also widely used as a scripting language because:
    1. It runs without compilation
    2. Executes line-by-line
    3. Ideal for automation tasks

4. How Python is interpreted?

- An interpreted language is any programming language which is not in machine-level code before runtime. Therefore, Python is an interpreted language.

  Python program runs directly from the source code. It converts the source code that is written by the programmer into an intermediate language, which is again translated into machine language that has to be executed.
  Because Python code is not directly compiled into machine code before execution.

- Instead, execution flow is:



- The interpreter reads and executes instructions at runtime.
  - Advantages:
    - Easier debugging
    - Platform independent
    - No compilation step
  - Disadvantage
    - Slightly slower than compiled languages

5. Is Python, case sensitive language or not?

- Yes, Python is a case-sensitive language. This means it treats uppercase and lowercase letters as distinct characters when resolving identifiers such as variable names, function names, class names, and keywords.

  For example:
    1. my_variable and My_Variable are considered two different variables.
    2. The built-in function print() must be written in all lowercase; using Print() will result in a NameError.
    3. Keywords such as if, else, and for must be used in their exact lowercase form.

       Understanding case sensitivity is essential for writing correct and predictable Python code. To handle situations where you might want case-insensitive comparisons (such as user

input), you can explicitly convert strings to a consistent case using methods like .lower() or .upper() before comparing them.

6. How memory is managed in Python?

- Python uses a private heap memory system managed internally by the Python memory manager.
    1. All Python objects and data structures are located in a private heap.
    2. The programmer does not have access to this private heap. The python interpreter takes care of this instead.
    3. The allocation of heap space for Python objects is done by Python's memory manager. The core API gives access to some tools for the programmer to code.
    4. Python also has an inbuilt garbage collector, which recycles all the unused memory and so that it can be made available to the heap space.
- Components involved
    - Memory Manager
        - Allocates memory for objects.
    - Garbage Collector
        - Automatically frees unused objects.
    - Reference Counting
        - Every object keeps track of references.
          When count = 0 → object deleted.
- 👉 Interview highlight line:
  Python primarily uses reference counting and a cyclic garbage collector to manage memory.

7. What are the key features of Python?

1. Python is an interpreted language. It means that, unlike languages like C and its variants, Python does not need to be compiled before it is run. Other interpreted languages include PHP and Ruby.
2. Python is dynamically typed, it means that you don't need to state the types of variables when you declare them or anything like that. You can do things like x=111 and then x="I'm a string" without error
3. Python is well suited to object orientated programming in that it allows the definition of classes along with composition and inheritance. Python does not have access specifiers (like C++'s public, private).
4. In Python, functions are first-class objects. This means that they can be assigned to variables, returned from other functions and passed into functions. Classes are also first class objects
5. Writing Python code is quick but running it is often slower than compiled languages. Fortunately, Python allows the inclusion of C-based extensions so bottlenecks can be optimized away and often are. The numpy package is a good example of this, it's really quite quick because a lot of the number-crunching it does isn't actually done by Python
6. Python finds use in many spheres – web applications, automation, scientific modeling, big data applications and many more. It's also often used as "glue" code to get other languages and components to play nice.

8. Why we create virtual environment in Python, venv vs Docker?

Python virtual environments (venv) and Docker containers serve similar goals of isolation and dependency management but operate at different levels of the system stack, making them suitable for different use cases. A venv isolates only Python dependencies, while Docker isolates the entire application and its operating system environment.

9. What is pickling and unpickling?

1. Pickle module accepts any Python object and converts it into a string representation and dumps it into a file by using dump function, this process is called pickling.
2. While the process of retrieving original Python objects from the stored string representation is called unpickling.

10. What is PEP 8?

- PEP stands for Python Enhancement Proposal. It is a set of rules that specify how to format Python code for maximum readability.
  PEP 8 is a coding convention, a set of recommendation, about how to write your Python code more readable.

11. Why indentation is required in Python?

- Indentation is required in Python because it is a mandatory part of the language's syntax used to define code blocks, unlike many other languages where it is only for style or readability. The Python interpreter uses the level of indentation to understand the structure and flow of the program.
- **Why Indentation is Mandatory**
  - **Defines Code Blocks:** Python does not use braces (`{}`) or keywords like `begin`/`end` to group statements. Instead, it uses indentation to determine which lines of code belong to a specific block, such as the body of a loop, conditional statement, function, or class.
  - **Controls Program Flow:** The indentation level dictates the execution flow. For example, in an `if` statement, only the indented lines immediately following the condition are executed if the condition is true.
  - **Enforces Readability and Consistency:** By making indentation mandatory, Python forces programmers to write clean, visually structured, and consistent code. This makes the code easier to read, understand, and maintain, especially in collaborative environments.
  - **Syntactic Rule:** If indentation is incorrect or inconsistent within the same block, the Python interpreter will raise an `IndentationError` or `TabError`, preventing the program from running.

- **Key Rules**
  - All statements at the same level of indentation belong to the same block.
  - A colon (`:`) typically signifies the beginning of an indented block (e.g., `if`, `for`, `def`, `class`).
  - The PEP 8 style guide (the official guide for Python code) recommends using **four spaces** per indentation level and avoiding the mixture of tabs and spaces within the same file to prevent errors.

12. What are Python namespaces?

- A namespace in python refers to the name which is assigned to each object in python. The objects are variables and functions. As each object is created, its name along with space(the address of the outer function in which the object is), gets created.

  A **namespace** in Python is a container (mapping) that holds names (identifiers) and the objects they reference. Internally, it behaves like a dictionary where key is the namespace and value is the address of the object:

  name → object reference

  Namespaces ensure that names don't conflict with each other by keeping them in separate contexts. When Python encounters a name, it searches for it in a specific order called the **LEGB rule.** Local → Enclosing → Global → Built-in.

- Sample example

```python
x = 10   # Global namespace

def func():
    x = 5   # Local namespace
    print(x)

func()
print(x)

# Output
5
10
# Python prints 5 first because it finds x in the local namespace before checking global
```

Level	Namespace	Meaning
L	Local	Inside current function, These namespaces are at the local or inner function.
E	Enclosing	Outer function (nested functions), These namespaces are at the higher level or outer function.
G	Global	Module-level names, These are namespaces for all the objects created at the level of the main program.
B	Built-in	Predefined Python names, These namespaces contain all the built-in objects in python and are available whenever python is running.

5. Programming CORE

1. Is it possible to return two or more values in Python?

- Yes, it is possible.
- Python functions can return multiple values by separating them with commas in the return statement (e.g., return a, b), which effectively returns them as a single tuple.

2. What is abstract in python?

- In Python, abstraction is an object-oriented programming (OOP) principle used to hide unnecessary implementation details and show only the essential features of an object or system to the user. This simplifies complex systems, improves modularity, and promotes code reusability.
- Abstraction is primarily achieved in Python using abstract classes and abstract methods via the built-in abc (Abstract Base Class) module.

1. Key Components of Abstraction in Python
    1. **Abstract Base Class (ABC):** A class that cannot be instantiated directly (you cannot create an object of an abstract class). It serves as a blueprint or template for other "concrete" classes, defining a common interface that all subclasses must follow.
    2. **Abstract Method:** A method declared within an abstract class but without an implementation (it has an empty body or simply passes). Subclasses are required to provide their own specific implementation for these methods; otherwise, a `TypeError` will be raised upon instantiation of the subclass.

    3. **Concrete Method:** Abstract classes can also contain regular methods with full implementations. These concrete methods can be inherited and used directly by subclasses, promoting code reuse of common functionality.
2. Purpose and Benefits
    1. **Enforce Structure:** Abstraction ensures that all derived classes follow a specific interface, leading to consistent and predictable code.
    2. **Reduce Complexity:** By hiding internal mechanisms, developers can focus on the higher-level logic of what the code does rather than how it works internally.
    3. **Facilitate Polymorphism:** It allows you to write code that works with objects of different classes that share a common abstract interface, making the code more flexible and adaptable.
3. How to Implement Abstraction
    1. To define an abstract class in Python, you must inherit from the `ABC` class and use the `@abstractmethod` decorator for methods that should be abstract.

```
from abc import ABC, abstractmethod

class Shape(ABC): # Inherits from ABC to be an abstract class
    @abstractmethod
    def area(self): # Abstract method, no implementation
        pass

    @abstractmethod
    def perimeter(self): # Abstract method, no implementation
        pass

    def describe(self): # Concrete method with implementation
        print(f"This shape has an area of {self.area()} and a perimeter of
{self.perimeter()}.")

class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self): # Must implement abstract method
        return self.length * self.width

    def perimeter(self): # Must implement abstract method
        return 2 * (self.length + self.width)

# shape = Shape() # This would raise a TypeError
rectangle = Rectangle(10, 5)
rectangle.describe() # Output uses the concrete method from the abstract
class
```

3. Mutable vs Immutable object difference

1. **Mutable Objects (e.g., lists, dicts):** Can be changed, updated, or modified after creation
2. **Immutable Objects (e.g., strings, tuples):** Cannot be altered once defined; any change creates a new object, ensuring data integrity.

4. What is Static method and Class methods and their differences.

- In Python, static methods are utility functions that have a logical connection to a class but do not need access to the class or instance data, while class methods are bound to the class and can access and modify class-level attributes using an implicit `cls` argument.
    1. Static Methods

- Decorator: Defined using the `@staticmethod` decorator.
- Implicit Arguments: Do not take any implicit first argument (like `self` or `cls`).
- Access to State: Cannot access or modify the class state or instance state directly within the method. They are essentially regular functions nested within a class's namespace.
- Usage: Used for utility functions, such as data validation or unit conversion, that logically belong to the class but are independent of specific instances or the class's overall state.

2. Class Methods
   - Decorator: Defined using the `@classmethod` decorator.
   - Implicit Arguments: Take the class itself as the first implicit argument, conventionally named `cls`.
   - Access to State: Can access and modify class-level attributes using the `cls` argument. They cannot directly access instance-specific data.
   - Usage: Commonly used for creating factory methods (alternative constructors) or operations that need to modify a class-level state that applies across all instances

Summary of Key Differences

Aspect	Class Method	Static Method
Decorator	@classmethod	@staticmethod
First Argument	`cls` (refers to the class)	No implicit argument
Access to Class State	Yes, can access/modify class attributes	No, cannot access/modify class attributes directly
Access to Instance State	No	No
Primary Use Case	Factory methods, modifying class state	Utility functions, code organization

5. Multiprocess, Multithreading, AsyncIO

- Multiprocessing achieves true parallelism for CPU-bound tasks using multiple CPU cores, while multithreading shares memory for I/O-bound tasks within one process. AsyncIO operates on a single thread using an event loop to handle non-blocking I/O tasks efficiently. Multiprocessing bypasses the Global Interpreter Lock (GIL), whereas threading is restricted by it.
  - **Multi-Processing:** Uses independent memory spaces and processes. Ideal for CPU-intensive tasks (e.g., heavy calculations, data processing).
  - **Multi-Threading:** Shares memory space within one process. Ideal for I/O-bound tasks (e.g., network requests, file reading).
  - **Async (Asynchronous):** Uses a single thread and event loop. Ideal for managing high-volume I/O tasks with less overhead than threads.
- **When to Use Which?**
  - Use **Multiprocessing** for intense computational work.
  - Use **Threading** for traditional I/O tasks.
  - Use **Async** for building responsive applications with many concurrent network connections.

```python
import asyncio
import time

async def say_after(delay, what):
    await asyncio.sleep(delay) # Non-blocking sleep, allows other tasks to run
    print(what)

async def main():
    print(f"started at {time.strftime('%X')}")

    # Create tasks to run concurrently
    task1 = asyncio.create_task(say_after(1, 'hello'))
    task2 = asyncio.create_task(say_after(2, 'world'))

    # Wait for both tasks to be completed
    await task1
    await task2

    print(f"finished at {time.strftime('%X')}")

if __name__ == "__main__":
    # Run the main coroutine
    asyncio.run(main())

# Output:
started at 17:14:32
hello
world
finished at 17:14:34
```

- **Key Differences Summary**

Feature	Multiprocessing	Multithreading	Async
Best For	CPU-bound	I/O-bound	I/O-bound (high concurrency)
Memory	Separate	Shared	Shared
Parallelism	True (Multi-core)	Concurrent (Single-core)	Concurrent (Single-core)
Overhead	High	Medium	Low
GIL Limitation	No	Yes (in Python)	No (Event loop)

6. A Global Interpreter Lock (GIL)

- Definition: The GIL is a mutex (or a lock) that allows only one thread to hold the control of the Python interpreter at a time. This means that even in a multi-core system, only one thread can execute Python bytecode at once.

  Key Points for Interviews:

Purpose: It was designed to make CPython's memory management simpler and to protect against race conditions in non-thread-safe C extensions.

- **Impact on Performance:** The GIL is a bottleneck for CPU-bound tasks (like heavy mathematical calculations). However, it does not significantly affect I/O-bound tasks (like web scraping or API calls) because the lock is released while waiting for the network or disk.

- Workarounds: To achieve true parallelism in Python, developers use the `multiprocessing` module (which creates separate memory spaces and interpreters for each process) rather than `threading`.

7. Debugging skills

1. Effective Python debugging involves a combination of a systematic approach and the use of various tools and techniques to identify, analyze, and resolve errors in your code.
    1. Core Debugging Techniques
        1. Reading Tracebacks
        2. Print Statements
        3. Logging
        4. Interactive Debugger (Pdb)
        5. Integrated Development Environment (IDE) Debugging
    2. Best Practices for Efficient Debugging
        1. Reproduce the Problem Consistently
        2. Break Down Complex Problems
        3. Write Tests
        4. Take Breaks
        5. Use Descriptive Error Messages
        6. Read the Source Code

8. Singleton class

- A **Singleton class** in Python is a class that guarantees only one instance of itself is created throughout the application's lifetime, providing a single, global point of access to that instance. Common use cases include managing database connections, logging, or configuration settings, where multiple instances could lead to resource conflicts or inconsistent behavior.
- While it can be implemented, many Python developers consider the classic singleton pattern an anti-pattern, often recommending module-level objects or dependency injection as more "Pythonic" alternatives that avoid issues with testing and global state.

9. Monkey patching

- Monkey patching in Python is a technique for dynamically modifying or extending the behavior of a module, class, or function at runtime without changing its original source code. This is possible due to Python's dynamic nature and mutable object model.

10. What is the difference between local and global variables?

- **Global Variables:** Declared outside of any function. They are accessible by any function within the same module.
- **Local Variables:** Declared inside a function's body. They are only accessible within that specific function and are destroyed once the function finishes execution.

11. How can you share global variables across modules?

- The most common way to share variables across modules in a project is to create a config or shared module:
    1. Create a file named config.py and define your variables there (e.g., x = 0).
    2. In other modules, import config.

3. Access or modify the variables using config.x.
4. Note: Using a dedicated module is cleaner than trying to manage globals across multiple individual scripts.

12. What is Keyword?

- Keywords are reserved words in Python that have special meanings to the interpreter. They define the syntax and structure of the language (e.g., if, else, while, def, class, import). You cannot use keywords as variable names, function names, or any other identifiers.

13. What is Float, int() function, print?

- Float: A data type used to represent "floating-point" numbers (numbers with decimal points, like 3.14).
- int() function: A built-in function used to convert a string or a float into an integer (it truncates decimals; int(3.9) becomes 3).
- print: A built-in function used to output text or variables to the console/terminal.

14. How can you convert a number to a string?

You use the built-in str() function.
age = 25
string_age = str(age) # Results in “25"

15. How to convert JSON into Python object

6. Python Fundamentals

1. Whenever Python exits, why isn't all the memory de-allocated

- When the Python interpreter exits, it attempts to destroy every object and de-allocate memory. However, some memory may remain allocated because:
  - **Circular References:** Objects that reference each other in a loop might not be cleared by the standard garbage collection cycle upon exit.
  - **Global Namespaces:** Objects referenced from global namespaces are not always freed.
  - **C Libraries:** Portions of memory reserved by C library extensions are impossible for the Python interpreter to de-allocate directly.
  - **OS Cleanup:** Python relies on its own cleanup mechanism, but the final cleanup of the heap is often left to the Operating System.

2. What is PYTHONPATH?

- PYTHONPATH is an environment variable that you set to tell the Python interpreter where to look for modules and packages. When you import a module, Python checks:
  - The directory containing the input script (or the current directory).
  - The directories listed in PYTHONPATH.
  - The installation-dependent default paths (standard library).

3. What are Assignment, Identity operators & Membership operators operators?

- **Relational operators** compare values
- **Assignment operators** assign values to variables
- **Identity operators** check if two variables refer to the same object in memory
- **Membership operators** test if a value exists within a sequence (e.g., list, string, or tuple)

- **RELATIONAL OPERATORS**
  - **Purpose:** Used to compare two operands to check for a relationship between them (e.g., equality, greater than, less than).
  - **Result:** They return a Boolean value (True or False).
  - **Examples:** == (equal to), != (not equal to), > (greater than), < (less than), >= (greater than or equal to), <= (less than or equal to).

```python
x = 10
y = 20
print(x == y) # Output: False (checks value equality)
print(x < y) # Output: True (checks if x is less than y)
```

- **ASSIGNMENT OPERATORS**
  - **Purpose:** Used to assign or update values of variables. They do not return a boolean result.
  - **Operators:** The primary operator is =, but there are also compound operators like +=, -=, *=, etc..

```python
x = 5 # Assigns the value 5 to the variable x
x += 2 # Adds 2 to x and updates x (x is now 7)
```

- **MEMBERSHIP OPERATORS (`IN`, `NOT IN`)**

  Membership operators are used to validate the presence of a value in a sequence or collection.
  - **in:** Returns True if the value is present in the sequence, False otherwise.
  - **not in:** Returns True if the value is not present in the sequence, False otherwise.
  - Example:

For dictionaries, in and not in check for the presence of a key, not a value.

```python
fruits = ["apple", "banana", "cherry"]
# Using 'in'
print("banana" in fruits)
# Output: True

# Using 'not in'
print("grape" not in fruits)
# Output: True
```

```python
x = [1, 2, 3]
y = x # y refers to the same object as x
z = [1, 2, 3] # z is a new object with the same value

# Using 'is'
print(x is y)
# Output: True (same object in memory)

print(x is z)
# Output: False (different objects, even with same value)
# Using 'is not'
print(x is not z)
# Output: True
```

- **IDENTITY OPERATORS (IS, IS NOT)**

  Identity operators compare the memory locations (object identity) of two objects to determine if they are the same object, not just if they have the same value. You can use the built-in id() function to see the unique memory address of an object.
  - **is:** Returns True if both variables point to the same object in memory, False otherwise.
  - **is not:** Returns True if both variables do not point to the same object in memory, False otherwise.

4. A What is the maximum possible length of an identifier?

- In Python, identifiers (names for variables, functions, classes, etc.) can be of any length. However, according to PEP 8 style guidelines, it is recommended to keep lines to a maximum of 79 characters, which logically encourages keeping identifier names descriptive but concise.

5. Property in Python.

6. Assembly

Compiled file: .pyc Executed by Python VM

7. What is Input? (The `input()` function)

- The input() function in Python is a built-in method used to get data from a user during the execution of a program. It pauses the program and waits for the user to type text on the keyboard and press the Enter key.

8. What is “if statement”?

- In Python, an if statement is a core conditional statement used for decision-making. It allows a program to execute a specific block of code only if a given condition evaluates to True.

9. For VS While Loop

Key Differences

Feature	For Loop	While Loop
Primary Use	Iterating over a sequence (list, string, range).	Repeating as long as a condition is True.
Iterations	Predefined (finite based on the iterable).	Uncertain (indefinite until condition changes).
Control	Automatically moves to the next item.	Requires manual update of the condition variable.
Infinite Loop	Rare; only if the iterable is infinite.	Common; occurs if the condition never becomes False.
Performance	Generally faster for sequences.	Slower due to repeated condition evaluation.

1. **The For Loop: Sequence-Based**
   A **for** loop is most "Pythonic" when you have a collection of items (like a list) and you want to do something with every item in that collection.

   Best For: Processing items in a list, reading every character in a string, or running a code block a specific number of times using range().

2. **The While Loop: Condition-Based**
   A while loop is used when you don't know exactly how many times the code needs to run in advance, but you know what condition should stop it.

Best For: Handling user input (e.g., "Keep asking until they type 'exit'"), waiting for a network connection, or any task where the end-point is dynamic.

3. Choosing Between Them
   1. Use **for** if you are "counting" things or going through a "bunch of stuff" you already have. Use **while** if you are "waiting" for something to happen or if the number of steps depends on external logic.

```python
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

```python
count = 0
while count < 3:
    print(count)
    count += 1  # Crucial: update the condition variable
```

10. What are Iterator & generators in Python?

1. **Iterators:** An object that can be iterated upon. It implements the `__iter__()` and `__next__()` methods. It remembers its state, so it knows which element is coming next.

   # Example of using an iterator
   my_list = [10, 20, 30]
   my_iterator = iter(my_list) # Get an iterator from an iterable

   print(next(my_iterator)) # Output: 10
   print(next(my_iterator)) # Output: 20
   print(next(my_iterator)) # Output: 30
   # next(my_iterator) would raise StopIteration

2. **Generators:** A special type of iterator created using functions and the yield keyword.
   The way of implementing iterators are known as generators. It is a normal function except that it yields expression in the function.

   Key Interview Distinctions:
   1. **Memory Efficiency:** Unlike a List (which loads all items into RAM), a Generator yields one item at a time. This is "lazy evaluation."
   2. **Use Case:** Ideal for processing massive datasets (like log files or database streams) where you cannot fit the entire dataset into memory.
   3. **State:** Once a generator is exhausted (you've iterated through all items), it cannot be re-used; it must be re-initialized

```python
def square_numbers(n):
    for i in range(n):
        yield i * i  # Pauses and returns a value each time

    # Using the generator
    squares = square_numbers(5)

    # Iterate through the generator (each iteration resumes the function)
    for num in squares:
        print(num)

square_numbers(5)
# Output:
# This will print the squares of numbers from 0 to 4:
# 0
# 1
# 4
# 9
# 16
```

11. What is the difference between Xrange and range?

- Xrange returns the xrange object while range returns the list, and uses the same memory and no matter what the range size is.

12. What is decorator with code examples? Meta-programming (Decorators)

- Decorators are a form of meta-programming used to modify or enhance the behavior of a function or class without changing its actual source code. They "wrap" another function.
  Decorators are used to add some design patterns to a function without changing its structure.
  Decorators generally are defined before the function they are enhancing.

  - Higher-Order Functions: In Python, functions are first-class objects. A decorator is a function that takes another function as an argument and returns a new function.
  - Syntax: Uses the `@decorator_name` "pie syntax" above a function definition.

```python
def my_decorator(func):
    def wrapper():
        print("Something before the function.")
        func()
        print("Something after.")
    return wrapper

@my_decorator
def say_hello():
    print("Hello!")
```

  - Use Cases: Common in enterprise applications for **Logging, Authentication/Authorization, Caching, and Timing execution**

- **Multiple decorator**

```python
def decorator_two(func):
    def wrapper():
        print("Decorator Two: Before function call")
        result = func()
        print("Decorator Two: After function call")
        return result
    return wrapper

def decorator_one(func):
    def wrapper():
        print("Decorator One: Before function call")
        result = func()
        print("Decorator One: After function call")
        return result
    return wrapper

@decorator_two
@decorator_one
def my_function():
    print("Original function body")
    return "Result"

my_function()

# Decorator Two: Before function call (outermost wrapper runs its "before" logic)
# Decorator One: Before function call (inner wrapper runs its "before" logic)
# Original function body (the actual function is executed)
# Decorator One: After function call (inner wrapper runs its "after" logic)
# Decorator Two: After function call (outermost wrapper runs its "after" logic)
```

13. Middleware & CORS

1. Middleware and CORS are crucial concepts for building secure and functional Python web APIs, particularly when separating the frontend (e.g., React, Vue) from the backend (e.g., FastAPI, Django, Flask).

   Middleware is a "software glue" that acts as a bridge between incoming requests and application logic, allowing for pre-processing of requests and post-processing of responses. CORS (Cross-Origin Resource Sharing) is a security feature enforced by browsers that restricts web pages from making requests to a different domain than the one that served them.

14. Middleware (Runs before/after request):

1. Middleware functions run for every request and response, enabling global actions without duplicating code in every route handler.
   1. **Functionality:** Used for logging, authentication, compression (GZip), session management, and modifying request/response headers.
   2. **Execution Order:** Middleware executes in the order it is added—first in for requests, last out for responses.
   3. **ASGI Middleware (FastAPI/Starlette):** These are asynchronous and manipulate the scope, receive, and send functions.
   4. **WSGI Middleware (Flask/Django):** These operate on the standard WSGI request/response cycle.

15. CORS Concepts (Cross-Origin Resource Sharing)

- Browsers implement the "same-origin policy," which prevents frontend applications on one domain (e.g., localhost:3000) from accessing APIs on another (e.g., localhost:8000).

- The Solution: CORS allows the server to explicitly define which origins are permitted to access its resources.

```python
# Example: Custom Middleware in FastAPI

from fastapi import FastAPI, Request
import time

app = FastAPI()

@app.middleware("http")
async def add_process_time_header(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request) # Process request
    process_time = time.time() - start_time
    response.headers["X-Process-Time"] = str(process_time) # Modify response
    return response
```

Preflight Requests: For "complex" requests (e.g., POST, PUT, DELETE, or custom headers), the browser sends an automatic OPTIONS request to verify permissions before the actual request.

CORS Headers: The server must respond with Access-Control-Allow-Origin, Access-Control-Allow-Methods, and Access-Control-Allow-Headers.

16. Implementing CORS in Python Frameworks

1. **FastAPI:** FastAPI uses CORS Middleware from starlette

2. **Flask:** Flask uses the flask-cors extension.

3. **Django:** Django requires the django-cors-headers library

```python
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware

app = FastAPI()

origins = [
    "http://localhost:3000",
    "https://myproductionapp.com",
]

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins, # Allows specific origins
    allow_credentials=True, # Allows cookies/auth headers
    allow_methods=["*"], # Allows all methods (GET, POST, etc.)
    allow_headers=["*"], # Allows all headers
)

@app.get("/")
def main():
    return {"message": "Hello World"}
```

17. Key Considerations & Best Practices

1. **Security:** Avoid allow_origins=["*"] in production, as it allows any website to access your API.
2. **Positioning:** Always add CORSMiddleware at the top of the middleware stack so it can handle preflight (OPTIONS) requests before other security checks block them.
3. **Authentication:** When using cookies or Authorization headers, allow_credentials must be True, and allow_origins must be specific (not *).
4. **Debug with Browser Tools:** CORS issues are always reported in the browser console, not the server logs.

18. A What is 'with' keyword in python?

- The with statement in Python simplifies resource management by automatically handling setup and cleanup, ensuring files or connections close safely even if errors occur.
    1. Replaces long try-except–finally blocks with cleaner syntax.
    2. Improves readability by reducing unnecessary boilerplate code.
    3. Example: Using with statement to Open a File:

       ```
       with open(...) as file
       ```

    4. Opens the file safely and assigns it to file. Inside the block, file.read() Reads the file content. When the block ends, Python automatically closes the file even if an error occurs.

       ```
       with open("sample.txt", "r") as file:
       data = file.read()
       print(data)
       ```

19. What is memory leaks, Diagnose and Fix Memory leaks

- Memory leaks in Python can occur when objects that are no longer being used are not correctly deallocated by the garbage collector. This can result in the application using more and more memory over time, potentially leading to degraded performance and even crashing. In this article, we will explore how to diagnose and fix memory leaks in Python.

20. How to Diagnose Memory Leaks in Python

- There are several tools that can be used to diagnose memory leaks in Python. Here are a few options:
  1. **Tracemalloc module:** The tracemalloc module is a built-in Python module that can be used to track the allocation of memory blocks in Python. It can be used to track the source code location where the memory was allocated, as well as the size of the allocated memory block. To use tracemalloc, you will first need to enable it by calling tracemalloc.start(). Then, you can take a snapshot of the current memory allocation by calling tracemalloc.take_snapshot(). This will return a Snapshot object, which contains information about the memory blocks that are currently allocated.

     You can then use the Snapshot.statistics() method to get a list of Statistic objects, which represent the memory blocks that are currently allocated, sorted by the size of the memory blocks. You can use this information to identify the source code locations where the largest memory blocks are being allocated, which may be indicative of a memory leak.

     Here's an example of how to use tracemalloc to take a snapshot and print the statistics:

```python
import tracemalloc

tracemalloc.start()

# Allocate some memory
a =  1] *  10 ** 6)
   [     (
snapshot = tracemalloc.take_snapshot()
top_stats = snapshot.statistics('lineno')

for stat in top_stats[:10]:
   print(stat)

# Output:
6a0aa145-b72e-42a5-a8b2-b92294e6e4d9.py:6: size=7813 KiB, count=2, average=3907 KiB
```

21. Async and await

- In Python, the `async` and `await` keywords are used to write concurrent code using coroutines within a single thread of execution, primarily with the built-in asyncio library.

  Key Concepts
  1. **async def:** This syntax defines an asynchronous function, also known as a coroutine function, which returns a coroutine object when called.
  2. **await:** This keyword is used only inside an async def function to pause the execution of the current coroutine until the "awaitable" operation (e.g., I/O operation, another coroutine, or a Future object) it is waiting on completes.
  3. **Event Loop:** The asyncio event loop manages and schedules the execution of multiple coroutines. When a coroutine encounters an await expression, it yields control back to the event loop, allowing other pending tasks to run in the meantime. This prevents the program from blocking during long-running I/O operations like network requests or file access.
  4. **Non-blocking I/O:** The core benefit is cooperative multitasking. While one task waits for an I/O operation to finish, the event loop can switch to another task that is ready to run, maximizing CPU usage.

     Basic Example

     To run asynchronous code, you typically use asyncio.run() as the main entry point (available

since Python 3.7)
As shown in the output, both "hello" and "world" are printed after approximately 2 seconds total, not 3 seconds (1 + 2), because they ran concurrently.

When to use async / await

Asynchronous programming is ideal for I/O-bound tasks where the program spends most of its time waiting for external operations, such as:

1. Network requests (e.g., with the aiohttp library).
2. Database operations.
3. File I/O operations

- It is less suited for CPU-bound tasks that require heavy computation, as Python's Global Interpreter Lock (GIL) still limits true parallel execution within a single process.

22. What is meant by jump statement?

- Jump statements in Python are used to alter the normal sequential flow of a program's execution by transferring control to another part of the code, such as exiting a loop or a function. The main jump statements in Python are break, continue, return, and pass.

23. What are the tools that help to find bugs or perform static analysis?

- PyChecker is a static analysis tool that detects the bugs in Python source code and warns about the style and complexity of the bug. Pylint is another tool that verifies whether the module meets the coding standard.

24. How are arguments passed by value or by reference

- Everything in Python is an object and all variables hold references to the objects. The references values are according to the functions; as a result you cannot change the value of the references. However, you can change the objects if it is mutable.

25. What is lambda in Python?

- It is a single expression anonymous function often used as inline function.
  square = lambda x: x*x

  Used when:
  - Small function needed
  - No reuse required
  - Inline operations

  Limitation:
  - Only one expression allowed.

26. Why lambda forms in python does not have statements

- A lambda form in python does not have statements as it is used to make new function object and then return them at runtime.

27. What is pass in Python?

- Pass means, no-operation Python statement, or in other words it is a place holder in compound statement, where there should be a blank left and nothing has to be written there.

28. What is docstring in Python?

- A Python documentation string is known as docstring, it is a way of documenting Python functions, modules and classes.

29. What is negative index in Python?

- Python sequences can be index in positive and negative numbers. For positive index, 0 is the first index, 1 is the second index and so forth. For negative index, (-1) is the last index and (-2) is the second last index and so forth.

30. How you can convert a number to a string?

- In order to convert a number into a string, use the inbuilt function str(). If you want a octal or hexadecimal representation, use the inbuilt function oct() or hex().

31. What is module and package in Python?

- In Python, module is the way to structure program. Each Python program file is a module, which imports other modules like objects and attributes. The folder of Python program is a package of modules. A package can have modules or subfolders.

32. Mention what are the rules for local and global variables in Python?

1. **Local variables:** If a variable is assigned a new value anywhere within the function's body, it's assumed to be local.
2. **Global variables:** Those variables that are only referenced inside a function are implicitly global.

33. How can you share global variables across modules?

- To share global variables across modules within a single program, create a special module. Import the config module in all modules of your application. The module will be available as a global variable across modules.

34. Explain how can you make a Python Script executable on Unix?

- To make a Python Script executable on Unix, you need to do two things, Script file's mode must be executable and the first line must begin with # (#!/usr/local/bin/python)

35. Explain how to delete a file in Python?

- By using a command os.remove (filename) or os.unlink(filename)

36. Explain how can you generate random numbers in Python?

- To generate random numbers in Python, you need to import command as
  import random random.random()
  This returns a random floating point number in the range [0,1)

37. Explain how can you access a module written in Python from C?

- You can access a module written in Python from C by following method,
  Module = =PyImport_ImportModule(“”);

38. How can you copy an object in Python?

- To copy an object in Python, you can try copy.copy( ) or copy.grnrcopy( ) for the general case. You cannot copy all objects but most of them.

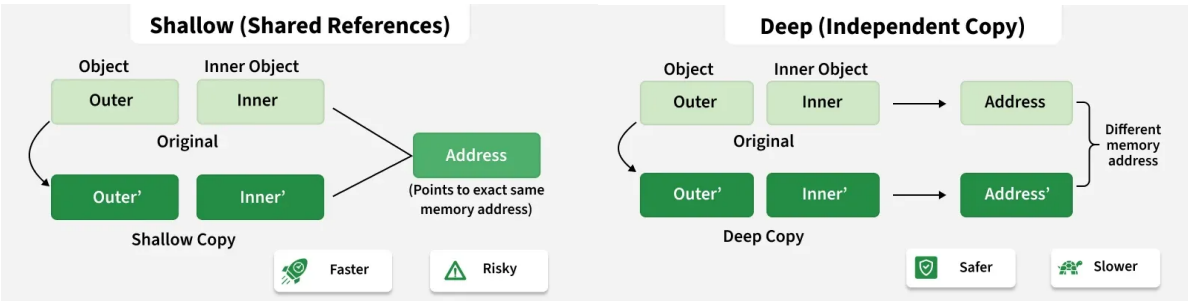
- The copy() method returns a shallow copy of the list.

  Why use it?
- If you just say list_b = list_a, they both point to the same spot in memory. If you change list_b, list_a changes too! Using list_b = list_a.copy() ensures that list_b is a separate "twin" that you can modify without hurting the original.

39. Shallow copy VS Deep copy using code example.

```python
import copy
original_list = [[1, 2], [3, 4]]

# Shallow Copy: creates a new top-level list, but inner lists are shared
shallow_copy = copy.copy(original_list)

# Deep Copy: creates a fully independent clone, including all inner lists
deep_copy = copy.deepcopy(original_list)

# Modify a nested element in both copies
shallow_copy[0][0] = 99
deep_copy[0][0] = 500

print(f"Original list: {original_list}")
Original list: [[99, 2], [3, 4]]
# The original list was affected by the change made to the shallow_copy because they both reference the same inner
# list object [1, 2]. However, it was not affected by the change to the deep_copy.

print(f"Shallow copy: {shallow_copy}")
Shallow copy: [[99, 2], [3, 4]]
# This is a new outer list, but the modification to its first nested element is reflected here and in the origina
#,confirming the shared reference.

print(f"Deep copy: {deep_copy}")
Deep copy: [[500, 2], [3, 4]]
# The deep copy is completely independent. The modification to its nested element does not affect the original lis
# or the shallow copy.
```

1. A shallow copy creates a new object that stores references to the original nested elements, meaning changes to mutable nested objects affect both the copy and the original. In contrast
2. A deep copy recursively duplicates all objects, ensuring the new object is completely independent of the original, with no shared references
   You should use a shallow copy when performance is critical and you don't need to modify



nested objects independently, and a deep copy when full independence is required to prevent unintended side effects.

Summary of Differences

Feature	Shallow Copy	Deep Copy
Level of Copying	Duplicates only the top-level container.	Recursively duplicates all objects, including nested ones.
Shared References	The original and copy share references to nested mutable objects.	No references are shared; the copy is fully independent.
Impact of Changes	Changes to nested mutable elements affect both the original and the copy.	Changes to the copy do not affect the original, even for nested elements.
Performance	Faster and uses less memory.	Slower and more memory-intensive due to recursive cloning.
Creation Methods	copy.copy(), list.copy(), [:] slicing, dict.copy() .	copy.deepcopy().

40. Mention the use of // operator in Python?

- It is a **Floor Division operator**, which is used for dividing two operands with the result as quotient showing only digits before the decimal point. For instance, 10//5 = 2 and 10.0//5.0 = 2.0)

41. Mention the use of the split function in Python?

- The use of the split function in Python is that it breaks a string into shorter strings using the defined separator. It gives a list of all words present in the string.
  Note: to distinguish ordinary words from function names, place a pair of empty parentheses after their names, even if the corresponding function wants one or more arguments. This is a standard convention.

  First, Python checks if the name specified is legal (it browses its internal data in order to find an existing function of the name; if this search fails, Python aborts the code)

  Second, Python checks if the function's requirements for the number of arguments allows you to invoke the function in this way (e.g., if a specific function demands exactly two arguments, any invocation delivering only one argument will be considered erroneous, and will abort the code's execution)

  Third, Python leaves your code for a moment and jumps into the function you want to invoke; of course, it takes your argument(s) too and passes it/them to the function

  Fourth, the function executes its code, causes the desired effect (if any), evaluates the desired result(s) (if any) and finishes its task;

  Finally, Python returns to your code (to the place just after the invocation) and resumes its execution.

  The backslash (\) has a very special meaning when used inside strings - this is called the escape character.

  1. If you want to put just one backslash inside a string, don't forget its escaping nature - you have to double it. For example, 1. an invocation like this will cause an error: 2. while this one won't:

42. Positional and Keyword Arguments

1. **Mixing:** Positional arguments must come before keyword arguments.
2. **Default Values:** Keyword arguments can utilize default values predefined in the function.

```
>>> print("\")
  File "<python-input-1>", line 1
    print("\")
          ^
SyntaxError: unterminated string literal (detected at line 1); perhaps you escaped the end quote?
```

```
>>> print("\\")
# Output
\
```

3. **Variable Arguments:**
   1. ***args** handles variable positional arguments, while
   2. ****kwargs** manages variable keyword arguments.
      default **end="\n"**, also we can use **sep="-"**

```
Positional Arguments: Values are assigned to parameters based on their position.
    func(arg1, arg2)
    complex(3, 5) (3 is real, 5 is imaginary).
Keyword Arguments: Values are assigned to parameters using the parameter name.
    func(name1=arg1, name2=arg2)
    complex(imag=5, real=3) (Order does not matter).
```

```
>>> print(sep="&", "fish", "chips")
  File "<python-input-0>", line 1
    print(sep="&", "fish", "chips")
                                   ^
SyntaxError: positional argument follows keyword argument
```

43. What is comment, its types?

- Python comments are statements that are not executed by the compiler. It is not considered as a part of program.

  There are two types of comment in Python.
  1. **Single line comment**
     It is used to comment only one line. Hash (#) is used for single line comment.
  2. **Multi line comment**
     It is used to comment a block of code. Triple quotes are used to multiline comment starts with ''' and ends with '''.

44. What is Keyword?

1. The word which is predefined in the library is called keyword. Keywords cannot use aEs a variable, function name, class name or any other identifier.

45. What is Float, int() function, print?

1. **float( )** is a predefined function which is used to convert given data into float.
2. **int( )** is a predefined function which is used to convert string into integer.

   It is a predefined function which is used to print data or information.

46. What is type conversion in Python?

- Type conversion (also known as type casting) is the process of converting a value from one data type to another.
  - Implicit Conversion: Performed automatically by Python (e.g., adding an integer to a float results in a float).
  - Explicit Conversion: Performed by the developer using built-in functions:
    - **int( ):** Converts to an integer.

- **float( ):** Converts to a floating-point number.
- **str( ):** Converts to a string.
- **list( ), tuple( ), set( ):** Converts to a sequence type.

47. Package VS Library

1. Package
    1. The package is a simple directory having collections of modules.
    2. This directory contains Python modules and also having __init__.py file by which the interpreter interprets it as a Package.
    3. The package is simply a namespace. The package also contains sub-packages inside it.
    4. Some Packages:
        1. Numpy
        2. Pandas
    5. Custom Package:

```
Student(Package)
 |        __init__.py (Constructor)
 |        details.py (Module)
 |        marks.py (Module)
 |        collegeDetails.py (Module)
```

2. Library
    1. The library is having a collection of related functionality of codes that allows you to perform many tasks without writing your code.
    2. It is a reusable chunk of code that we can use by importing it into our program, we can just use it by importing that library and calling the method of that library with a period(.)
    3. However, it is often assumed that while a package is a collection of modules, a library is a collection of packages.
    4. Some Libraries:
        1. Matplotlib
        2. Pytorch
        3. Pygame
        4. Seaborn etc.
    5. Example:
        1. Importing pandas library and call read_csv method using an alias of pandas i.e. pd.

```
import pandas as pd
df = pd.read_csv("file_name.csv")
```

48. What is first class objects in Python?

- A **first-class object** (also known as a first-class citizen or entity) in a programming language is a program entity that is handled uniformly and without restriction throughout the language.

  This means a first-class object must possess the following key properties, much like other fundamental data types (such as integers and strings):

- Can be assigned to a variable.
- Can be passed as an argument to a function or procedure.
- **Can be returned** as the result of a function call.
- **Can be stored in data structures** like lists, arrays, or hash tables.
- Can be created at runtime.

7. LIST TUPLE DICTIONARY AND SET

1. What are the built-in data type does python provides

1. Python provides several built-in data types which are categorized as
   1. Mutable (changeable after creation)
      List [ ] Sets ( ) Dictionaries { }

   2. Immutable (cannot be changed after creation)
      Tuples ( ) Strings “” Numbers

2. What is the difference between list and tuples in Python?

Feature	List	Tuple	Set	Dictionary
Syntax	[ ]	( )	{ }	{"key": "value"}
Mutable (editable)	Yes	No	Yes	Yes
Ordered	Yes	Yes	No	Yes (3.7+)
Duplicates	Allowed	Allowed	Unique	Unique Keys
Access	Indexing	Indexing	Methods	Key
Speed	Lists are slower than tuples	Tuples are faster than lists.		
Hashable	No	Yes		

3. When to use tuple?

- fixed data - dictionary keys - performance critical code

4. Why Tuple is immutable?

1. Because immutability provides:
   - data safety - faster performance - hashability - thread safety

   Tuples are implemented as fixed-size arrays internally, making them faster and memory efficient.

5. Can we add an elements in tuple, why, how?

1. You cannot directly add elements to an existing tuple because tuples are immutable. This means that once a tuple is created, its size and contents cannot be changed in place.
   1. Concatenation: You can use the + operator to combine the original tuple with a new one (or a single-element tuple for a single item). This creates a new tuple object.

      You can also use the augmented assignment operator +=, which is a shorthand

for original_tuple = original_tuple + new_element. This still results in the creation of a new tuple object in memory.

2. Type Conversion (Tuple to List and Back)
    1. For adding multiple elements or inserting at a specific position, the most flexible approach is to convert the tuple to a list, use list methods (like append( ) or insert( )), and then convert the list back to a tuple

```python
original_tuple = ("apple", "banana", "cherry")

# Convert to a list
temp_list = list(original_tuple)

# Use list's append method
temp_list.append("orange")

# Convert back to a tuple
updated_tuple = tuple(temp_list)

print(updated_tuple)
# Output: ('apple', 'banana', 'cherry', 'orange')
```

```python
original_tuple = ("apple", "banana")

# The comma is essential for a single-element tuple
new_element = ("cherry",)

# Create a new tuple by concatenation
updated_tuple = original_tuple + new_element

print(updated_tuple)
# Output: ('apple', 'banana', 'cherry')
```

6. List, dict, comprehension

1. Lists and dictionaries are fundamental data structures, while comprehension is a concise and efficient syntax for creating them (along with sets and generators) from existing iterables.

    1. **List**
    An ordered collection of items, which can be of different data types. Lists are mutable, meaning their elements can be changed after creation, and they are defined using square brackets [ ]

    Use Cases: Ideal for maintaining sequences of items, functioning as stacks (LIFO) or queues (though collections.deque is more efficient for queues), and generally when the order of elements matters.

    Example: [1, "apple", 3.14]

    2. **Dictionary (Dict)**

    An unordered collection of key-value pairs, where each key is unique and immutable (e.g., strings, numbers, or tuples), and values can be of any type. Dictionaries are mutable and are defined using curly braces {} with a colon separating keys and values.

    Use Cases: Used for fast look-ups and retrieval of data based on a key (average constant time complexity O(1)), and mapping data points, such as mapping usernames to user data.

    Example: {"name": "Alice", "age": 25, "city": "New York"}

    3. **Comprehension**

    A powerful, single-line syntax for creating new data structures (lists, dictionaries, sets, or generators) by iterating over an existing iterable and applying an expression and optional filtering conditions.

```python
squares = [x**2 for x in range(10) if x % 2 == 0]
# Result: [0, 4, 16, 36, 64]

squares_dict = {x: x**2 for x in range(5)}
# Result: {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}
```

Advantages: Comprehensions are often more readable, concise, and generally faster than traditional for loops for creating new collections.

Types: Python supports list, dictionary, set, and generator comprehensions.

- **List Comprehension Syntax -** Uses square brackets [ ] and returns a list.
- **Dictionary Comprehension Syntax -** Uses curly braces { } and requires both a key and a value expression separated by a colon.

7. Dictionary Questions:

1. I have a dictionary it contains set of value, key is 1 and value is active and key is 2 value is inactive and last key is 3 value is proactive. I want to delete 1 key without changing original dictionary.
2. If I want to get dictionary value using key but that key doesn't present in dictionary.

8. File Handling & Exceptions

1. What is file handling in python?

- File handling in Python is the process of managing files on your computer's storage system, allowing programs to create, open, read, write, and close files. This enables data to be stored permanently, rather than just in memory while a program is running.

2. What is file operation in python? (Opening, Writing, Appending, Reading, Closing)

1. Opening: The open(filename, mode) function is used to create a file object (or "file handle") that serves as a link to the file, specifying the intended operation via the mode argument.
2. Reading: Retrieving data from a file is done in read mode ('r') using methods like read( ), readline( ), or readlines( ).
3. Writing: Adding new data to a file is done in write mode ('w') using the write( ) or writelines( ) methods. This mode will overwrite the file's content if it already exists; otherwise, it creates a new file.
4. Appending: Adding new data to the end of a file without deleting existing content is done in append mode ('a') using the write() method.
5. Closing: The close() method is used to release system resources associated with the file and ensure all data is saved.

3. What is read ( ) function in file handling?

- The read() function is a common method used in file handling across many programming languages (including Python, C, and C++) to access and retrieve content from an opened file

4. How do you open a file for writing? (The old list had "how to delete," but not opening/writing)

- To open a file for writing in Python, you use the built-in open() function with a specific mode, most commonly using the with statement for automatic file closure.

5. What are Reasons of Exception?

- Exceptions in Python occur during the execution of a program when an unexpected situation or error condition arises, disrupting the normal flow of instructions

6. What is try, except, finally block?

1. The try, except, and finally blocks in Python are used for exception handling, a mechanism to manage runtime errors gracefully without crashing the program.
2. Here is a breakdown of each block:
   1. **try**
      1. This block contains the code that might cause an error (exception) during execution.
      2. If an exception occurs within the try block, the remaining statements in the block are skipped, and control is transferred to the appropriate except block.
   2. **except:**
      1. This block defines the code to run if an exception is raised in the try block.
      2. You can specify the type of exception to catch (e.g., ZeroDivisionError, ValueError) for tailored error handling, or use a general except to catch any exception.
      3. The goal is to handle the error in a controlled manner (e.g., print a user-friendly message, log the error) so the program can continue running or fail gracefully.
   3. **finally:**
      1. This block contains code that will always execute, regardless of whether an exception occurred in the try block, was caught by an except block, or the block was exited via a return, break, or continue statement.
      2. It is typically used for essential cleanup actions, such as closing files, releasing system resources (like network connections or database connections), or logging, to ensure resources are properly managed and memory leaks are prevented.
   4. The try block in Python is a segment of code used for testing for errors (exceptions) that might occur during execution. It is the fundamental part of Python's exception handling mechanism, which prevents the entire program from crashing when an unexpected error happens.

7. Explain important Python errors? (Arithmetic Error, Import Error, Index Error)

Python includes several built-in exceptions to handle different kinds of errors. Here is an explanation of the ArithmeticError, ImportError, and IndexError.

1. **ArithmeticError:** The ArithmeticError is the base class for exceptions that occur during arithmetic operations, meaning it's a general category rather than a specific error you raise directly. More specific errors like ZeroDivisionError, OverflowError, and FloatingPointError inherit from it.

2. **ZeroDivisionError:** This is the most common ArithmeticError and happens when you try to divide a number by zero.
   - Example: 10 / 0

3. **OverflowError:** This occurs when an arithmetic operation results in a number too large to be represented by the specific numeric type being used (though this is less common in modern Python with its arbitrary precision integers, it can still happen with specific data types or operations).

4. **ImportError:**
   1. An ImportError occurs when a Python program is unable to load a module. This can happen for several reasons:
   2. Module Not Found: The most common reason is that the module you are trying to import does not exist in the paths Python searches. Example: import non_existent_module
   3. Name Mismatch: You might have a file named the same as a standard library module, causing a conflict.
   4. Syntax Errors in the Module: If the imported module itself has a syntax error, the ImportError (or a subclass, ModuleNotFoundError in Python 3.6+) will be raised during the import attempt.

5. **IndexError**

- The IndexError is raised when you try to access an index that is outside the bounds of a sequence, such as a list, tuple, or string. Python uses zero-based indexing, so a list with 5 elements has valid indices from 0 to 4.
  1. Accessing an Out-of-Bounds Index: Attempting to use an index that does not exist. Example: python

     ```
     my_list = [10, 20, 30]
     print(my_list[5]) # This will raise an IndexError
     ```

  2. Empty Sequence: Trying to access the first element of an empty list (e.g., [] [0]) will also result in an IndexError.

9. UNIT TEST

- Pytest interviews commonly feature questions about its core features like fixtures, parametrization, and test organization. Below are common interview questions categorized by experience level.

1. Beginner-Level Questions

1. What is Pytest and why is it used?

1. It is a popular, open-source Python framework for writing and running unit, integration, and end-to-end tests. Key benefits include simple syntax, robust features (like fixtures), and automatic test discovery.
2. **How do you write a simple test case in Pytest?** Create a file starting with `test_` and define functions within it, also starting with `test_`, using the `assert` keyword to verify results.
3. How do you run tests using the command line? Use `pytest` for all tests, `pytest <file_name.py>` for specific files, or `pytest <file_name.py>::<function_name>` for individual tests.
4. **What is an assertion in Pytest?** It uses the built-in `assert` keyword to check conditions and provides detailed reports upon failure.
5. **What is the `pytest.ini` file used for?** It is a configuration file to customize Pytest's behavior, such as setting test discovery rules, adding command-line options, or registering markers.

2. Intermediate-Level Questions
   1. **Explain the concept of a fixture in Pytest and how to define one.** Fixtures provide a fixed baseline for tests, handling setup and teardown tasks. They are defined using `@pytest.fixture` and passed as arguments to tests.
   2. **What are fixture scopes, and what are the available options?** The `scope` parameter determines how often a fixture runs. Options range from `function` (default), `class`, `module`, `package`, to `session`.
   3. **What is the purpose of the `conftest.py` file?** It holds shared fixtures and hooks for multiple test files, with automatic discovery by Pytest.
   4. **How do you parametrize a test in Pytest?** Use the `@pytest.mark.parametrize` decorator to run a single test function with multiple input sets.
   5. **What is the difference between `skip` and `xfail`?**
      1. `@pytest.mark.skip` completely skips a test, while `@pytest.mark.xfail` marks a test that is expected to fail.

3. Advanced-Level Questions
    1. How do you handle mocking or monkeypatching in Pytest? Use the built-in `monkeypatch` fixture to safely replace dependencies, or the `pytest-mock` plugin for a `mocker` fixture.
    2. How do you run tests in parallel? Use the `pytest-xdist` plugin with the `-n` option (e.g., `pytest -n 4`).
    3. How do you generate a code coverage report? Use the `pytest-cov` plugin with `pytest —cov=my_module_name`.
    4. **Explain the purpose of `yield` in a fixture.** It divides setup (code before `yield`) from teardown (code after `yield`), essential for managing resources like database connections.

4. What is unit test in Python?

- A unit testing framework in Python is known as unit test. It supports sharing of setups, automation testing, shutdown code for tests, aggregation of tests into collections etc.
-

10. Lists & Sequences

1. What is slicing?

- A mechanism to select a range of items from sequence types like list, tuple, strings etc. is known as slicing.

  Slicing extracts part of sequence.

  ```
  s = "python"
  s[1:4]  # yth
  ```

  Syntax:

  ```
  [start : stop : step]
  ```

2. What is pop ( ) and clear ( ) function in list? Removing Elements

1. **pop(index):** Removes and returns the item at the given index. If you don't provide an index, it removes the last one. It's great when you need to "use" the item you just removed.
2. **clear( ):** This is the "nuke" option. It removes every single item from the list, leaving you with an empty list [ ]

3. What is sort( ) vs reverse ( ) vs count ( ) function in list?

1. **sort( ):** Sorts the list in place (usually ascending). Note: It doesn't return a new list; it changes the original one!
2. **reverse( ):** Flips the order of the list in place. The last becomes first, and the first becomes last.
3. **count(value):** Returns the number of times a specific value appears in the list.

4. What is append( ) vs extend( ) vs insert ( ) function in list?

Method	What it does	Example
append( )	Adds a **single** element to the very end of the list.	[1, 2].append(3) → [1, 2, 3]
extend( )	Unpacks an iterable (like another list) and adds **each** item to the end.	[1].extend([2, 3]) → [1, 2, 3]
insert( )	Adds an element at a **specific index**.	list.insert(0, "Hi") adds "Hi" at the start.

5. How can you count duplicate elements in a given list?

1. If you want to count how many times *every* item repeats, the most "Pythonic" way is using `collections.Counter`.

```python
from collections import Counter

my_list =  'a', 'b', 'a', 'c', 'b', 'a']
counts = Counter(my_list)

# Output
# Counter({'a': 3, 'b': 2, 'c': 1})
```

11. Libraries

1. **SQLAlchemy(ORM):** The Python Powerful SQL toolkit and Object-Relational Mapper that maps Python classes to database tables
2. **Pydantic:** Data validation and settings management library using Python type hints, crucial for FastAPI.
3. **Alembic:** Lightweight database migration tool designed for usage with SQLAlchemy.
4. **Uvicorn / Gunicorn:** Lightning-fast ASGI server (Uvicorn) and WSGI application server (Gunicorn) to run Python web applications.
5. **pylint / ruff:** Pylint performs in-depth code linting; Ruff is an extremely fast Python linter and formatter.
6. **pytest:** Mature framework for writing compact, readable, and scalable tests in Python.
7. **pytest-cov:** Plugin for Pytest to measure code coverage, showing which lines were executed during tests.
8. **pytest-asyncio:** Pytest support for testing asynchronous (async/await) Python code.
9. **pytest-mock:** Thin-wrapper around unittest.mock to easily mock or patch objects in tests.
10. **pytest-cases:** Separates test case data (parameters) from test logic for cleaner, parameterized testing
11. **pytest-xdist:** Enables parallel test execution to significantly speed up test suites.
12. **pytest-html:** Generates HTML reports for Pytest results to share detailed test summaries.

12. Big Libraries

1. **NumPy(numerical python):** Nympy is a python package, written for scientifica computing and data analysis.
    1. Multi dimensional array objects
    2. Often used along with packages like SciPy(Scientific Python) and Matplotlib(Plotting Library).
    3. N-Dimension Array
        1. The most important object defined in Numpy, ndarray
        2. it describes the collection of items of the same type
        3. Items in the collection can be accessed using a zero based index
        4. Every item in an nd array takes the same size of the block in the memory. each element in ndarray is an object of data type object(called dtype)
2. **Pandas:** A data manipulation and analysis library offering data structures like DataFrames and tools for data cleaning, transformation, and analysis.
3. **Matplotlib:** A comprehensive library for creating static, interactive, and animated visualizations in Python.
4. **Seaborn:** A statistical data visualization library based on Matplotlib, providing a high-level interface for drawing attractive and informative statistical graphs.
5. **Scikit-learn:** A simple and efficient tool for data mining and data analysis, featuring various classification, regression, and clustering algorithms.
6. **PyTorch:** An open-source machine learning framework that accelerates the path from research prototyping to production deployment, known for tensor computation with strong GPU acceleration.
7. **OpenCV (Open Source Computer Vision Library):** A vast collection of algorithms and functions aimed at solving computer vision problems in real-time applications.

13. Tools

1. **PIP -** pip is the package installer for Python. You can use pip to install packages from the Python Package Index and other indexes
2. **PyPI -** The Python Package Index (PyPI) is a repository of software for the Python programming language.
3. **Conda -** Conda is an open source package management system and environment management system for installing multiple versions of software packages and their dependencies and switching easily between them. It works on Linux, OS X and Windows, and was created for Python programs but can package and distribute any software.
4. **Jupyter Notebook -** The Jupyter Notebook is the original web application for creating and sharing computational documents. It offers a simple, streamlined, document-centric experience.
5. **MongoDB and DynamoDB -** MongoDB and DynamoDB are both prominent NoSQL databases, but they excel in different areas:
    1. MongoDB offers superior flexibility for complex queries and diverse data structures (BSON/documents), making it ideal for flexible, self-hosted or Atlas-managed applications.
    2. DynamoDB, a fully managed AWS service, is better suited for high-traffic, low-latency applications, providing seamless auto-scaling and cost-effective performance, but with limited, key-value-focused queries.

14. Python Fields

1. Programming with Python
    1. Python fundamentals, exploratory statistics, data quality
    2. Python Object And Data Structures Operations
    3. Python Statements
    4. UDF Functions And Methods
    5. File And Exception Handling
    6. Python Modules And Packages
    7. Object Oriented Programming
    8. Advanced Python Modules
    9. Package Installation And Parallel Processing

10. SQL (Structured Query Language)

2. Framework

1. Rest API With Flask And Python
2. Rest API Integration With Databases For Web App Development
3. Data Wrangling With Python And Exploratory Data Analysis

3. Data Analysis

1. Data Analysis Using Advance Excel
2. Data Analysis With Python
3. Data Analysis Using Numpy

4. Data Visualization

1. Matplotlib, Seaborn, Art of Storytelling, Plotly, Bokeh

5. Database / Big Data SQL language, Data Integration

6. Business Intelligence

1. Power BI
2. Introduction to Business Intelligence, Make, Tableau, Looker Studio

7. Advanced Databases

1. Advanced SQL
2. Cloud Fundamentals
3. Snowflake
4. AWS CP

8. Extraction and management of Data

1. Text Mining
2. Web Scraping with BeautifulSoup
3. Selenium

9. Machine Learning

1. Introduction to Models and Methodology

15. Miscellaneous & Coding Snippets

1. What is import method?
2. Explain the output of the following piece of code? (Tuple multiplication example)

```
tup = (1, 2)
result = tup * 3
print(result)
```

3. What does the following code give us? (Tuple type checking example)
4. What will the following code output? (String slicing example)
5. What is string in python?
6. What if you want to toggle case for a Python string? (`swapcase()`)
7. Give example of sleep() function in Python
8. How can you Get the Google cache age of any URL or web page?
9. What is map function in Python?
10. Multiple Choice Questions (length of identifier, private variables, floor division, etc.)

16. JPMC Interview Questions

1. 4. JPMC, 27th Feb 2026

1. Find bugs in below program

```
async def a ( )
        r1 = b ( )
        r2 = c ( )
        result = r1 + r2
        result = r3(result)
        return result
r = a()
```

2. Given an array of integers **nums** and an integer **k** target, return the total number of subarrays whose sum equals to k

1. A sub-array is contiguous non-empty sequence of elements within an array.
    1. **Input :** nums = [1,1,1], k = 2
       Output : 2
       Explanation: Subarrays: [1,1] [1,1] have sum equal to 2
    2. **Input :** nums = [1,2,3], k = 3
       Output : 3
       Explanation: Subarrays: [1,2] [3] have sum equal to 3
    3. **Input :** arr[ ] = [10, 2, -2, -20, 10], k = -10
       Output : 3
       Explanation: Subarrays: arr[0...3], arr[1...4], arr[3...4] have sum equal to -10.
    4. **Input :** arr[] = [9, 4, 20, 3, 10, 5], k = 33
       Output : 2
       Explanation: Subarrays**:** arr[0...2], arr[2...4] have sum equal to 33.
    5. **Input :** arr[] = [1, 3, 5], k = 2
       Output : 0
       Explanation: No subarrays with 0 sum.
3. What is Dependency Injection in FastAPI

2. 3. JPMC, 9th Jan 2026

1. Tuple versus List using code, why Tuple is immutable, Can we add an elements in tuple, why?
2. Shallow copy versus deep copy using code example.
3. What is decorator with code examples?
4. What is Static method and class methods and there differences.
5. Property in Python.
6. What is with keyword in python?
7. What is abstract in python?
8. Linked List and detect cycle and list

3. 2. JPMC, Dot NET., 18th Dec 2026

1. Agile
2. Constructors
3. Access modifiers
4. Polymorphism
5. Abstraction
6. Encapsulation
7. Inheritance
8. Normalization
9. 10 GB chunks
10. Decorators
11. Middleware
12. Assembly
13. reverse string
14. Second highest element from an array

4. [Java] Ashwini

1. Find duplicate occurrences, how many number divisible by given number
2. Kadane`s

5. [Java] Poornima

1. [2,3,-8,7,-1,2,3] Output = 11

6. Shared by Neha

1. Data types and data structures
2. Diff b/w list and tuple
3. Generator
4. Decorator
5. Iterator
6. Diff Range and xrange

7. List, dict, comprehension
8. Remove duplicates or find duplicate?
9. Singleton class
10. Monkey patching
11. List comprehension
12. What is memory leak
13. How to convert json into python object
14. Which package is used for reading xl file
15. I have a dictionary it contains set of value, key is 1 and value is active and key is 2 value is inactive and last key is 3 value is proactive. I want to delete 1 key without changing original dictionary.
16. If I want to get dictionary value using key but that key doesnt present in dictionary.
17. Create a model student with name and location and save data into database using django
18. Multiple decorator
19. Async and await
20. How to make generator
21. How to find file in a folder in python?
22. Generator that gives square of number till 5 ?

7. Python Programs

1. Flat the array = [1,2,3,[4,5,6,[7,8,9,10]]]
2. Fibonacci
3. Find duplicate occurrences / Remove duplicates?
4. How to find file in a folder in python?
5. Generator that gives square of number till 5 ?
6. Common elements from list. **Input [1,1,2,2,2] => Output [(1,2),(2,3)]**
7. I have a list [1,2,3,4,5] list 2 [4,5,6,7,8] Create a list where it will be combination of list and unique elements
8. Check two strings are anagram or not?
9. Count vowels in string
10. Max element from list
11. 10 GB chunks

```
def read_chunks(file):
  with open(file) as f:
      while True:
          data = f.read(1024*1024)
          if not data:
              break
          print(data)
```

12. How many number divisible by given number [x for x in arr if x%k==0]
13. Reverse string
    s[::-1]
14. Second highest element from an array
    sorted(set(arr))[-2]
15. Queue First-In, First-Out (FIFO), Stack LIFO(Last In First Out) or FILO(First In Last Out), Make stack behaving like a queue

8. Most Imp Topics

1. List, Tuple, Dictionary, Generator, Iterator, Lambda function, Comprehension

17. Agents

Provider	Model name
OpenAI	GPT-4.1, GPT-5 to GPT-5.2
Anthropic	Claude Haiku, Opus, Sonnet 4, 4.5
Google	Gemini 2.5 Pro, Gemini 3 Flash, Gemini 3 Pro
Grok xAI	Code Fast 1
Raptor mini	Fine - tuned GPT-5 mini